

Tejas — Slides 1 & 3

Slide 1 — Opening

"Good [morning/afternoon] everyone. I'm Tejas, and together with Ameya and Prathmesh, we're working on a fraud detection system for online marketplaces.

The project is called Seller-Level Fraud Detection using Interpretable Machine Learning. The two things that make this project interesting are right there in the name — interpretability, and the fact that we're operating at the seller level, not just the transaction level. Prathmesh will get to that part.

To set the scale — we're working with a dataset of 299,695 transactions across 6,000 unique users. The average user has about 50 transactions in the dataset. And critically, only 2.2% of those transactions are actually fraudulent — that's 6,612 fraud events out of nearly 300,000 records. That number drives every decision we've made in this pipeline, and you'll see why as we go through the slides. I'll now walk you through the data and what we've built so far."

Slide 3 — Feature Enrichment Pipeline

"So let me walk you through exactly what we've done with the data.

We loaded the dataset — 299,695 rows, 17 columns — and the first thing we checked was data quality. Zero nulls across every column. The dataset is clean, which let us go straight into EDA.

From that analysis, we derived three features that we've already built into the pipeline.

The first is country_match — a binary flag for whether the transaction country matches the BIN country, which is the country of the card's issuing bank. About 92% of transactions have a match, meaning 8% don't — and that mismatch is a meaningful fraud signal.

The second is night_flag — we extracted the hour from the transaction timestamp and flagged transactions between 10pm and 5am. Our EDA confirmed that hours 23, 1, and 2 have the highest fraud rates in the dataset — around 2.4 to 2.5%.

The third is log transform on the amount. The raw amount has a mean of 177 but a standard deviation of 307 — heavily right-skewed. After log transform, the standard deviation drops to 1.15, which is much better behaved for models like Logistic Regression and SVM.

We also confirmed from the correlation matrix that shipping_distance_km has the strongest correlation with fraud at 0.27, and amount at 0.20. AVS and CVV flags are key binary signals we're preserving.

Finally, we've locked in our train-val-test split — 60/20/20, fully stratified. All three splits preserve the 2.2% fraud rate exactly. I'll hand it to Ameya now."

Ameya — Slides 2 & 4

Slide 2 — Moving Beyond the Black Box

"Thanks Tejas. So we've validated the data and we know what we're working with. Now let me explain why existing approaches aren't enough — because that's what motivated our design.

Two problems. The first is the regulatory constraint. Most high-performing fraud models today are XGBoost or deep learning based. They can be accurate, but they're black boxes. A financial regulator or a marketplace compliance team can't act on a score alone — they need to know why a transaction was flagged. Without that explainability, the model can't be responsibly deployed.

The second is the evasion problem. Our EDA actually shows this directly — web channel transactions have a fraud rate of 3.6%, compared to only 0.8% on the app channel. That's a 4x difference. A fraudster who knows this will spread their activity intentionally to stay below detection thresholds. Single-transaction flagging misses that behavioral pattern entirely.

Our architecture addresses both — SHAP and LIME for interpretability, and a seller-level aggregation layer for the evasion problem. But first, let me walk you through the ML engine we're building."

Slide 4 — Three-Pillar Architecture

"Our pipeline is structured around three pillars.

The first is the Imbalance Strategy. At 2.2%, a model that predicts no fraud for everything hits 97.8% accuracy but catches nothing. We will compare five strategies — class weights, undersampling, SMOTE, threshold tuning, and probability calibration — and empirically show which combination best improves recall without collapsing precision. We won't just pick one and hope.

The second is the Modeling Array. We will train six model families — Logistic Regression, k-NN, Naïve Bayes, Decision Trees and Random Forest, XGBoost and LightGBM, and SVM. The goal isn't just to find the best performer — it's to surface the accuracy-interpretability tradeoff across families. Logistic Regression gives us calibrated probabilities and full explainability. XGBoost likely wins on raw AUC. We want to quantify that gap.

The third is the Interpretability Framework. We will apply global feature importance, permutation importance, SHAP at the model level, LIME at the individual transaction level, and partial dependence plots for our top drivers — which we already know from EDA are shipping distance and amount. The goal is that for any flagged transaction, we can produce a human-readable explanation of what drove the risk score.

I'll hand it to Prathmesh for the piece that ties it all together."

Prathmesh — Slides 5 & 6

Slide 5 — The Seller-Level Risk Layer

"Thanks Ameya. Everything you've seen so far operates at the transaction level — a transaction comes in, gets a fraud probability. That's table stakes.

What we're building on top of that is what makes this project different.

Look at the scatter plot on the left. Each dot is a transaction. Some high-risk ones fall below the detection threshold — they look fine individually. But if you look at the seller those transactions belong to, the pattern becomes obvious.

So after the model scores each transaction, we will aggregate those scores into a Seller Entity Profile with three signals.

The first is the Average Risk Score — a rolling window of recent predicted probabilities for that seller. The second is the High-Risk Fraction — the percentage of that seller's transactions that breach the safety threshold. The third is the Risk Trend Slope — a velocity metric. Is this seller's fraud risk getting worse over time, or staying flat?

The motivation for this comes directly from our EDA. We found that fraud isn't uniformly distributed — certain channels, certain merchant categories, and certain time windows show concentrated risk. A seller operating heavily in travel on the web channel at night is a very different risk profile than an app-based grocery seller. The seller layer captures that cumulative behavioral signature.

The bottom line is — we're not just flagging bad transactions. We're identifying bad actors."

Slide 6 — Trajectory and Next Steps

"So concretely, here's where we stand.

What's done: the dataset is loaded and validated — zero nulls confirmed. EDA is complete — we have fraud rates by channel, by merchant category, and by hour of day. Three engineered features are live in the pipeline: country_match, night_flag, and log-transformed amount. And our stratified 60/20/20 split is locked in — the 2.2% fraud rate is preserved in all three splits.

What we will do next — Phase 1 is completing the feature engineering and running SMOTE experiments. Phase 2 is training the full modeling array with cross-validation and hyperparameter tuning. Phase 3 is deploying the SHAP and LIME layer on top of the best-performing models. Phase 4 is implementing the seller-level aggregation we just described.

The final deliverable will be a fully interpretable, multi-layered fraud detection system — not just something that works on a leaderboard, but something a real fraud analyst could actually use in production.

We're happy to take any questions."

Quick reminders before you go in:

- **Tejas** — lean on the real numbers. Zero nulls, 0.27 correlation for shipping distance, 92% country match rate. Those came from your code and they're specific — specificity builds credibility.
- **Ameya** — the 3.6% vs 0.8% web vs app fraud rate from your EDA is a great concrete hook for the evasion problem. Use it on slide 2, it makes the motivation feel real rather than theoretical.
- **Prathmesh** — be clear on what's built vs. what's planned on slide 5. The seller layer is designed and motivated by the EDA, but implementation is Phase 4. Don't overstate it, the professor will ask.